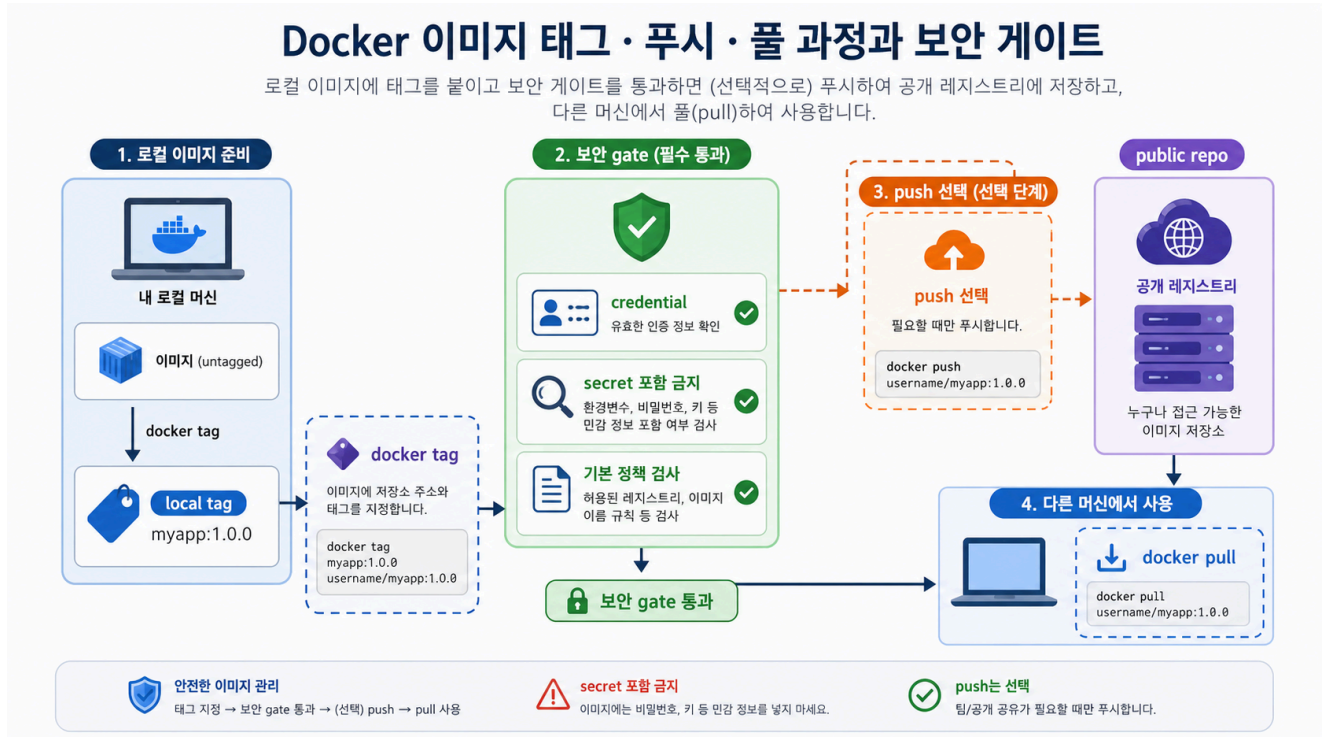


# 7교시: Tag/digest/registry gate - image 공유 기준 판단하기



## 수업 목표

- tag, digest, registry의 역할을 구분한다.
- tag가 새 build가 아니라 image reference 추가일 수 있음을 확인한다.
- version, latest, env, build id, git sha 같은 tag 기준을 상황별로 선택한다.
- Docker Hub push를 credential/security gate 이후 선택으로 판단한다.

## 개념 설명

Tag는 사람이 읽기 쉬운 image 이름표다. paperclip-static-site:day3 같은 tag는 협업과 수업 진행에 편하지만, tag는 바뀔 수 있다. 같은 tag가 항상 같은 content를 보장한다고 생각하면 안 된다.

Digest는 registry에서 image content를 식별하는 값이다. 재현 가능한 배포에서는 tag와 digest를 함께 보는 이유가 여기에 있다. Registry는 image를 저장하고 공유하는 장소다. local image는 내 컴퓨터에만 있지만, registry에 push하면 다른 machine, CI runner, Kubernetes가 pull할 수 있다. 그래서 push 전에는 public 범위와 secret 포함 여부를 반드시 확인한다.

Tag를 정할 때는 먼저 누가 이 이름을 보고 무엇을 결정하는가를 물어야 한다. 개발자는 debug와 추적을 원하고, 운영자는 배포 대상과 rollback 기준을 원하고, 보안 담당자는 어떤 source와 scan 결과에서 온 image인지 확인하고 싶어 한다. 그래서 하나의 image에도 여러 tag가 붙을 수 있다.

## Tag 적용 기준

| 기준                  | 예시                         | 적합한 상황                                     | 주의할 점                                                                     |
|---------------------|----------------------------|--------------------------------------------|---------------------------------------------------------------------------|
| application version | 1.4.2, v1.4.2              | 웹 애플리케이션 릴리즈와 image를 맞출 때                  | 회사마다 semantic version, calendar version, sprint/release train 정책이 다를 수 있음 |
| environment         | dev, staging, prod         | 환경별 배포 흐름을 사람이 빠르게 볼 때                     | 같은 tag가 계속 바뀌므로 재현성 기준으로 단독 사용하지 않음                                       |
| latest              | latest                     | local demo나 기본 pull 편의가 필요할 때              | 운영 배포 기준으로 쓰면 어떤 build인지 흐려짐                                              |
| build number        | build-128, ci-20260619-128 | CI 실행 번호와 image를 연결할 때                     | application version 없이 build number만 있으면 제품 릴리즈 의미가 약함                    |
| git sha             | sha-a1b2c3d                | source commit 추적과 rollback evidence가 필요할 때 | 사람이 읽기 어렵기 때문에 version tag와 함께 쓰는 편이 좋음                                   |
| release candidate   | 1.4.2-rc.1                 | QA, staging, pre-release 검증                | prod 승격 시 어떤 정식 version으로 이어졌는지 남겨야 함                                     |
| review/status       | day3-reviewed, scan-passed | 수업/검수 상태를 표시할 때                            | 상태 tag는 시간이 지나면 사실과 달라질 수 있으므로 scan note와 함께 관리                           |

Version tag는 가능하면 웹 애플리케이션의 버전과 맞춘다. 회사마다 version을 붙이는 방식은 다르다. 어떤 회사는 v1.4.2처럼 semantic version을 쓰고, 어떤 회사는 2026.06.19 같은 calendar version을 쓰고, 어떤 회사는 release train이나 sprint 번호를 쓴다. 그래도 image version이 애플리케이션 version과 따로 놓면 장애 분석 때 이 container가 어떤 앱 릴리즈인가를 다시 추적해야 한다. 기본 원칙은 \*\*웹 애플리케이션 릴리즈 버전이 image tag의 중심이 되고, git sha/build number/env tag는 보조 tag로 붙인다\*\*는 것이다.

같은 image에 여러 tag를 붙일 수 있다.

```
docker tag paperclip-static-site:day3 paperclip-static-site:v1.0.0
docker tag paperclip-static-site:day3 paperclip-static-site:sha-demo123
docker tag paperclip-static-site:day3 paperclip-static-site:staging
```

이때 v1.0.0은 앱 릴리즈를 말하고, sha-demo123은 source 추적을 돕고, staging은 현재 환경 배포 대상을 말한다. 세 tag가 모두 같은 image를 가리킬 수 있지만 의미는 다르다.

## Tag 선택 예시

| 상황          | 권장 tag 조합                         | 이유                         |
|-------------|-----------------------------------|----------------------------|
| 수업 실습       | day3, day3-reviewed               | 학습 단계와 검수 상태를 빠르게 확인       |
| 웹앱 정식 릴리즈   | v1.4.2, sha-<short>, 필요 시 prod    | 앱 버전, source, 환경을 분리해서 추적  |
| staging 검증  | v1.4.2-rc.1, sha-<short>, staging | 정식 전 후보와 commit 추적         |
| CI 임시 build | build-128, sha-<short>            | pipeline 결과와 source를 연결    |
| 운영 배포       | v1.4.2와 digest pin                | tag 가독성과 digest 재현성을 함께 확보 |

## 실습 명령

```
docker tag paperclip-static-site:day3 paperclip-static-site:day3-reviewed
docker tag paperclip-static-site:day3 paperclip-static-site:v1.0.0
docker tag paperclip-static-site:day3 paperclip-static-site:staging
docker images paperclip-static-site
docker image inspect paperclip-static-site:day3-reviewed --format "{{json
.RepoTags}}" {{json .RepoDigests}}"
```

Expected:

```
paperclip-static-site    day3
paperclip-static-site    day3-reviewed
paperclip-static-site    v1.0.0
paperclip-static-site    staging
```

day3-reviewed, v1.0.0, staging은 모두 같은 image id를 가리킬 수 있다. 중요한 것은 tag가 많다는 사실이 아니라 각 tag의 의미를 설명할 수 있는가다.

## Optional push gate

Docker Hub push는 선택이다.

1. public/private repository 범위를 설명할 수 있는가?
2. `.dockerignore`와 context 점검으로 secret 포함 위험을 줄였는가?
3. tag가 앱 버전, 환경, 빌드 출처, 검수 상태 중 무엇을 표현하는지 설명할 수 있는가?
4. password/token/MFA가 README, screenshot, terminal output에 남지 않는가?
5. version tag는 웹 애플리케이션의 실제 릴리즈 버전과 맞는가?

## 판단 기준

| 항목         | 좋은 상태                                                |
|------------|------------------------------------------------------|
| tag        | v1.0.0, sha-<short>, staging, day3-reviewed처럼 목적이 보임 |
| version    | image tag가 웹 애플리케이션 릴리즈 버전과 연결됨                      |
| latest     | 편의 tag로만 보고 운영 재현성 기준으로 단독 사용하지 않음                   |
| env        | dev/staging/prod가 현재 배포 대상임을 알되 불변 version으로 착각하지 않음 |
| provenance | base image와 custom build 출처를 설명 가능                   |
| digest     | registry에서 content 식별 기준으로 확인 가능                     |
| push       | gate 통과 시 선택, 아니면 local tag까지만 수행                    |

## 핵심 포인트

registry는 공유를 쉽게 하지만 실수도 공개한다. Day 3에서는 push 자체보다 push해도 되는 image인지 판단하는 gate가 더 중요하다.

## 다음 연결

마지막 교시는 build/run/check/cleanup과 failure RCA를 README handoff로 정리한다.